

Framed Cases

Backend AI Engineering portfolio copy built from real project decisions and outcomes.

VOICE CARD

Direct, technical, clear, grounded, no buzzwords

Audience

AI Engineering hiring managers and technical collaborators who want evidence of how I approach difficult technical work.

One action

Invite me to interview for an AI Engineering role, or contact me about a technically relevant collaboration.

Selected work

- Agentic-Nexus - multi-agent / RAG system
- TensorFlow - Python 3.14 free-threaded open-source work
- Webometrics - ranking consistency research

Agentic-Nexus

A local-first multi-agent RAG system built to learn how routing, retrieval, execution, and evaluation work together end to end.

The problem

I wanted to go beyond a single chatbot or a basic RAG pipeline and understand how a more complete agentic system behaves when several components have to coordinate. The point was learning by building: make routing explicit, connect different retrieval paths, control failure modes, and test the whole flow as one system.

What I did

I built the workflow as a routed multi-agent system rather than one fixed chain. A manager produces a structured routing plan instead of passing free-form instructions between agents. I used LAG-style schema/grammar-constrained generation so routing decisions behave like a state-machine output rather than prose that another component has to interpret. Retrieval combines FAISS vector search, a NetworkX knowledge graph, and DuckDB records. A CAG cache checks repeat queries before retrieval, while CRAG confidence scoring decides whether local context is enough or whether the workflow should fall back to web search. The final answer passes through an evaluator, with one bounded retry when evaluation fails. The system uses local models through Ollama and includes LangSmith tracing/evaluation support.

What came of it

I tested the pipeline end to end and verified that the routing paths, retrieval flow, direct-answer path, and evaluator loop worked as intended. I do not present it as a production benchmark because I did not keep formal performance logs. I present it as a working engineering prototype that taught me how to design, connect, inspect, and control a multi-agent AI workflow instead of treating agents as a single prompt wrapped in code.

Evidence

Public repository: github.com/MinaIbrahim10/Agentic-Nexus

TensorFlow Open-Source Contribution

Python 3.14 free-threaded compatibility, concurrency safety, and source-build work in TensorFlow.

The problem

TensorFlow was not fully ready for Python 3.14 free-threaded mode. Some code paths could bring back assumptions around the GIL after TensorFlow was imported, while other areas exposed concurrency risks such as deadlocks, unsafe critical-section behavior, and race conditions. Supporting Python 3.14t also meant the build and dependency infrastructure had to recognize and build against the free-threaded interpreter correctly.

What I did

I worked across TensorFlow's Python/C++ boundary and build setup instead of treating the work as one isolated patch. I fixed native-extension compatibility issues, removed or corrected assumptions that depended on the GIL, worked through critical-section and synchronization problems, investigated deadlock and race-condition paths, and updated Python 3.14t build/dependency support. I repeatedly rebuilt TensorFlow from source and tested the resulting library against the free-threaded interpreter so changes were validated in the actual runtime they were meant to support.

What came of it

The work led to successful TensorFlow source builds and testing on Python 3.14 free-threaded, and multiple upstream pull requests from this effort were merged into TensorFlow. The contributions covered compatibility, concurrency safety, and build support. Through this work I became a TensorFlow open-source contributor, with experience working inside a large production ML codebase where a small runtime change can affect C++, Python bindings, build tooling, tests, and thread-safety assumptions at the same time.

Webometrics Ranking Research

A data-driven audit of whether published university rankings are internally consistent with their underlying indicators and peer history.

The problem

A final university rank can look authoritative even when its relationship to the underlying indicators is not obvious. I wanted to test whether unusual ranking outcomes could be identified systematically instead of relying on visual inspection or isolated examples. The goal was not to label an institution suspicious because one model said so; it was to build a methodology that checks consistency from more than one direction.

What I did

I built a research pipeline around live and historical Webometrics data. I crawled the ranking at full scale, assembled historical editions, normalized institutions across time, and compared published rank behavior with what the indicator structure and comparable peers would suggest. During the work I also corrected weaknesses in my own early approach: I stopped extrapolating from only the top slice of the ranking, removed circular exclusions of elite institutions, corrected the discrepancy direction, replaced fixed-contamination anomaly flags with objective thresholds, and moved toward percentile, z-score, historical, and peer-calibrated evidence. The method became stronger when multiple signals had to agree instead of allowing a single anomaly detector to decide the story.

What came of it

The project developed from exploratory analysis into a published research paper and a reproducible auditing approach. More important than the publication itself, it forced me to handle the full research engineering loop: large-scale crawling, historical data cleaning, ranking normalization, anomaly analysis, methodological debugging, and turning ambiguous results into a method I could defend.

Bio and Contact

Bio

I am an AI Engineer focused on building and debugging systems end to end. My work spans multi-agent and RAG architectures, LLM workflows, machine learning, backend APIs, and open-source engineering. I am most interested in problems where model behavior, software design, evaluation, and runtime constraints all matter at the same time. I prefer work I can test, inspect, and explain over demos that only look impressive on the surface.

Contact / CTA

If you are hiring for an AI Engineering role and want someone who can work across model behavior, backend systems, evaluation, and debugging, I would be glad to talk. I am also open to technically serious collaborations where there is a real system to build or a difficult problem to investigate.

Before / After

Generic draft	Edited version
"Results-driven AI engineer passionate about leveraging cutting-edge technologies to build innovative, scalable solutions that create impact."	"I build AI systems end to end, and I care about the parts that are easy to hide in a demo: routing decisions, retrieval quality, evaluation, runtime behavior, and whether the system still works when assumptions break."

Why the edit works

The generic version could describe almost anyone. The edited version points to the kind of engineering work shown in the cases above and uses language I can stand behind in an interview.

What this document covers

Voice card: Five words that define the tone used throughout the cases.

Three framed cases: Every Selected Work item from the sitemap has a problem, what I did, and what came of it.

Specific project detail: Each case uses technical choices and outcomes that belong to that project rather than generic portfolio language.

One audience / one action: The copy is aimed at AI Engineering hiring managers and technical collaborators, with a clear interview/collaboration CTA.

Bio and CTA: Reusable portfolio copy is included in the same voice.

Before / after: A generic line is shown next to the edited version to make the voice difference visible.

Editing rule

Keep only claims I can explain and defend. Cut any line that sounds impressive but is not supported by the work.